



Slipstream: Architecture Options for Real-Time Process Analytics

Christian Janiesch
SAP Australia Pty Ltd
7/52 Merivale St
South Brisbane, QLD 4101
+61 7 3259 9617
c.janiesch@sap.com

Martin Matzner
University of Muenster
Leonardo-Campus 3
48149 Münster
+49 251 83-38088
matzner@ercis.de

Oliver Müller
University of Muenster
Leonardo-Campus 3
48149 Münster
+49 251 83-38064
mueller@ercis.de

Robert Vollmer
SAP AG
Dietmar-Hopp-Allee 16
69190 Walldorf
+49 6227 7-66985
robert.vollmer@sap.com

Jörg Becker
University of Muenster
Leonardo-Campus 3
48149 Münster
+49 251 83-38100
becker@ercis.de

ABSTRACT

Timely insight into a company's business processes is of great importance for operational efficiency. However, still today companies struggle with inflexibility of monitoring solutions and reacting to process information on time. We review the current state of the art of business process management and business activity management. Based on that we develop an architecture for event-driven business activity monitoring which is capable of standalone process analytics or can make use of complex event processing engines to filter and aggregate process events in real-time. We discuss advantages and disadvantages of the two options in terms of ease of use, flexibility, and extensibility and close by introducing on future research directions based on our experiences from a prototypical implementation using standard software.

Categories and Subject Descriptors

C.0 [General]: Modeling of computer architecture, System architectures.

General Terms

Management, Measurement, Documentation, Performance, Design, Standardization.

Keywords

Architecture, complex event processing, business process management, business activity monitoring.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

SAC'11, March 21-25, 2011, TaiChung, Taiwan.

Copyright 2011 ACM 978-1-4503-0113-8/11/03...\$10.00.

1. INTRODUCTION

The value creation of an enterprise, be it a manufacturer or a service provider, or a business network lies in its ability to execute its processes on time, on budget, and with the expected outcome. Hence, the processes performance is of the highest importance to an enterprise's success [7]. However, business process analytics is often not able to provide insight in a timely manner as data is only provided by extraction, transformation, and loading jobs batched over night similar to financial data. This data provisioning concept does not allow for real-time insight into processes which is necessary for some use cases, such as health care processes, logistics processes with perishable goods or just-in-time production processes to name a few. It is necessary to expose process information that resides in the wake of a current process execution: the processes slipstream.

By law, enterprise systems have to provide some support to log their transactions. Consequently, most process engines produce a process log as well which allows tracing its process instances. This entails that a huge amount of data is already available which could be used to gain insight into processes.

We propose to loosely couple systems through business events which we expose out of the process log. Business events represent state changes in process execution. We extend upon a simple proof of concept by using a complex event processing (CEP) engine. Hence, with an event-driven architecture, it is not only possible to observe process information from multiple servers and display it in real-time but it is also possible to initiate actions as a result of the analysis which exceed simple alerting capability.

We describe an architecture which is based on the ideas of BPM and CEP which is scalable and flexible so that it can also be used in a networked scenario. The architecture provides a starting point for the design concrete of *business activity monitoring* (BAM) solutions. But it can potentially cover more elaborate scenarios.

The structure of the paper is as follows: In the subsequent section, we perform a quick review of related work in business process management, business process analytics, and business activity

monitoring in particular. We detail our architecture in Section 3 and discuss two implementation options. The architecture enables real-time business activity monitoring which is flexible, extensible, and scalable across a process landscape. We conclude with a summary and comparison and state immediate next steps and future research issues.

2. RELATED WORK

Processes are generally seen as activities performed within or across companies or organizations [14]. Activities are considered to be the smallest entity used to build a process. These entities may be seen as complex chunks, which do not need to be analyzed further, or atomic activities, which cannot be divided any further. Furthermore, an activity can either refer to work, which needs to be done manually, or to work, which can be done automatically. Business Process Management (BPM) refers to a collection of tools and methods for achieving an understanding and then for managing and improving an enterprises' or business network's process portfolio [18; 21]. It plans, controls, and monitors intra- and inter-organizational processes with regards to existing operational sequences and structures in a consistent, continuous, and iterative way of process improvement [1]. A BPM system or engine allows for the definition, execution, and logging of business processes.

Business process analysis investigates if the actual behavior of single process instances is compliant with the prescribed business process design. Single process instances can be analyzed through behavior data that informs on the state-of-operation of processes' constituting activities. Generally, this means that processes are analyzed with focus on the behavior of completed processes, currently running process instances, or predicting the behavior of process instances in the future [22]. Each of these perspectives corresponds to a certain set of methods that is yet to be established in practice [5; 15; 22].

Business Activity Monitoring (BAM) strives to support the real-time monitoring of currently active business processes' behavioral properties which is our main objective. BAM systems focus on capturing and processing events with minimum latency [13] and therefore have to be closely linked with operational data sources. BAM foremost strives for the monitoring of key operational business events for changes or trends indicating opportunities or problems, and enabling business managers then to take corrective actions [13]. Single events recorded are usually not very meaningful in a BAM context, "instead many events have to be summarized to yield so-called key performance indicators" (KPI) [4]. BAM systems are then typically engaged with updating KPI [11].

Such KPI can be distinguished into basic KPI and aggregated KPI consisting of basic KPI [9]. Enterprises might use KPI to track organizational effectiveness, implementing a top-down performance management approach [12].

Therefore a BAM system must be able to detect events occurring in enterprise systems relevant to the activity monitored, calculate information for temporal processing, integrate event and contextual business information on the fly delivering quality data, execute business rules to set thresholds according to key performance indicators and other business-specific triggers, and

provide intuitive interfaces for presenting rules and metrics [2; 13].

A typical BAM solution therefore comprises of four components: an event processing module, a process definition module, a monitoring module, and a visualization module [2; 3].

In contrast, functionality offered by BAM tools in practice is limited to rather simple performance indicators such as flow time and utilization [16]. In parts, the BAM concept goes beyond tracking KPI. Instead, software systems contain some event processing functionality, namely "filtering, transformation, and most importantly aggregation of events" [5]. By applying rules engines to KPI, BAM systems can generate alerts and actions which inform managers of critical situations to possibly alter the behavior of the running process [22]. "This can be done in batch mode [...] or in online mode so that the current value of the KPI can be continually tracked, typically on a dashboard" [5].

3. SLIPSTREAM ARCHITECTURE

3.1 Architecture Options

The major event producers in our scenario are BPM systems themselves. For analytic purposes, two components of a BPM system are of main interest [for a complete overview on components of a BPM system cf. e.g. 19].

The process engine provides an environment for the execution of process instances, based on existing process definitions. The process monitoring component offers functions to track the progress of single process instances and to log corresponding audit data including data on occurred state transitions of process-relevant objects (e.g. processes, activities, resources, work items). This audit data constitutes a rich source of low-level process events for subsequent analysis, but is usually not accessible from outside the BPM system.

In order to enable real-time analytics on business processes or any system for that matter one needs to be able to detect state changes in the system which is to be monitored in the first place. After first conceptual designs and after analyzing the structure of common BPM software such as jBPM and SAP NetWeaver BPM, we concluded that tapping into the log of audit data is the best option for a non-invasive approach on the process models themselves.

While it is obviously possible to gather most of the information necessary for monitoring by adding additional tasks or events to a process model in a notation such as BPMN, it is inconvenient as every model has to be changed accordingly. The log of a BPM engine usually contains all state changes in a process such as *process_started*, *task_started*, *task_ended*, or *process_ended*. It does not necessarily cover all desired information, but in order to extend existing software without implementing a fully functional internal event bus, the log can serve as a reasonable source of information. Hence, we used log information in the proof of concept of our architecture. Since BPM engines sometimes run in a reduced logging level to decrease resource utilization and consequently do not log sufficient information, other sources might need to be considered.

Hence, a third component – an event publisher – is required to make the data available for external systems for processing and visualization. With that, two general ways of implementation are possible. Figure 1 shows both of them in an architecture diagram.

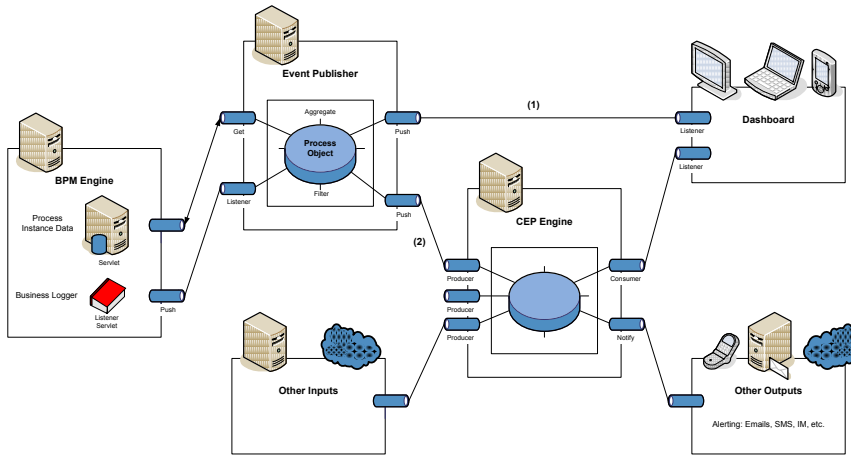


Figure 1: Implementation Options.

Option (1) introduces an event publisher to not only expose the events to the outside but also to already filter and aggregate them so that they can be sent to a visualization frontend such as a dashboard right away. The event engine can be standalone or part of the BPM engine. Option (2) involves using a complex event processing (CEP) engine which can pick up these published events, either raw or pre-aggregated, and work with them. While option 1 caters for simple scenarios and involves less application development, option 2 can provide a solution to much more complex scenarios and even extend beyond monitoring.

3.2 Slipstream Standalone

Following option 1 is a lean approach to BAM which online involves a slight extension of the BPM engine and some aggregation and filtering by an event publisher which can be as simple as a small servlet running on a web server.

The main challenge is to enable event publisher capability. An event publisher is capable of sniffing, formatting, and sending events [10].

In the sniffing step, the adapter detects events in the process audit data and other relevant sources (e.g. work item lists). Typically, the sniffer maintains a list of filters that defines which events should be observed and subsequently extracted to prevent sensible data from being published or simply to reduce the volume of events. Such filters can be defined on type level (e.g. only events signifying that an activity has been *completed* shall be extracted; events of type *running* and *cancelled* shall be filtered out) or instance level (e.g. only events with a process definition *ID* from 4711 to 4742 shall be extracted). It is important that event sniffing must be benign, i.e. it must not change the target system's behavior.

As mentioned above, virtually all BPM engine have a component to log process state changes. Consequently, the following general implementation options for an output adapter are possible. Cf. Figure 2 for an overview.

The logging engine usually employs a *Log Entry Manager* to persist the log entries from the *Process Engine* in a *Log* database (*WriteLogEntry*). The functionality of this manager can be extended to notify a listener (*NotifyListener*) which can package

an event (*LiveReporting*) and send it out to a monitoring component (*Slipstream Core*).

After detecting an event, the corresponding data has to be extracted and transformed to a specific event format. Most BPM systems use proprietary formats for event representation in their log. Recently, there have been attempts to standardize event formats, e.g. the XML-based Business Process Analytics Format (BPAF) [20]. However, not all system events can be easily mapped to such a standard format. BPAF, for example, deals only with state changes in processes and activities. Other process-related entities such as applications and work items are not covered. Another issue might be the availability or accessibility of event attributes. For example, not all BPM systems log the

relationship between a process instance and an application that is launched by that process instance permanently. Also, some information might be implicit and is not persisted in any log and, thus, not exposed as an event, e.g. the size of a user's work list.

Of course, the format is of lesser importance if the monitoring component is internal as in this scenario. It does however make sense to plan for extensibility as monitoring scenarios tend to grow.

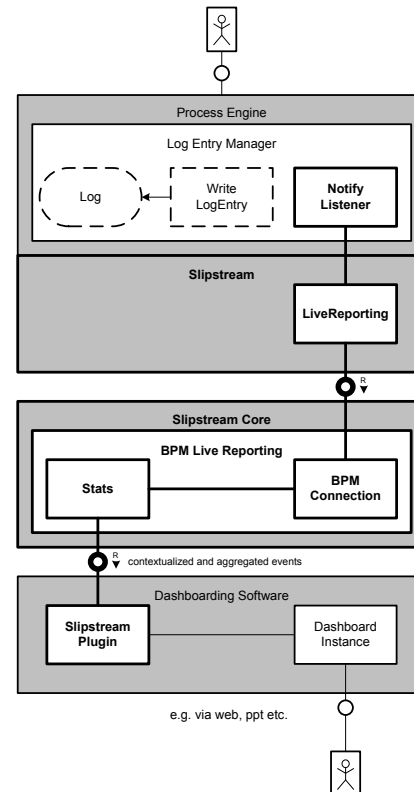


Figure 2: Implementation Option *Slipstream Core*.

As introduced above, the Slipstream Core receives the event stream in a format it is capable of interpreting for *BPM Live Reporting* through its *BPM Connection* component. A BPM connection translates the incoming events into a format that can

be understood by subsequent components of the core application. In a standalone implementation contextualization can be done either by a *Stats* package for aggregation and filtering or – to a lesser degree – by active dashboards which are capable of aggregation themselves. It is more convenient and better reusable to bundle this functionality in one statistical component which then sends complex events to *Dashboarding Software*. The complex events then already include aggregated/ derived data such as individual process runtimes, sums thereof or counts.

“A dashboard is a visual display of the most important information needed to achieve one or more objectives; consolidated and arranged on a single screen so the information can be monitored at a glance” [6]. Dashboards provide high-level summaries of information in a concise, clear, intuitive way and often in real-time [6]. This real-time aspect makes dashboards especially appealing in context of event-driven business activity management, as it fits the concept of event streams. Typical information to be displayed in a process dashboard include KPI (e.g. runtime, idle time), occurred or anticipated exceptions (e.g. when certain thresholds of KPI are or will likely be exceeded), or visualizations of the actual flow/ path of process instances. The visualized information is often averaged on the basis of the last n events of an event output stream or on the basis of a certain time window.

The *Dashboard Instance* can pick up the (complex) events either through a standardized plug-in or a specialized plug-in for the server (*Slipstream Plugin*). The communication works similarly to the already described principles for BPM engine to Slipstream Core. The dashboard can be made available through business intelligence portal software, presentation software, or general collaboration platforms.

3.3 Slipstream CEP

As mentioned above, it is also possible to use standard technology for the event routing and aggregation which comes with its own advantages and disadvantages. In the following, we describe how complex event processing (CEP) can be applied to monitor processes in real-time.

Complex event processing (CEP), in general, comprises a set of techniques for making sense of events in a system by deriving higher-level knowledge from lower-level system events in a timely and online fashion [4; 5; 10].

The central concept in CEP is the event. In everyday use, an event is something that happens. In CEP, an event is defined as a data object that is a record of an activity that has happened, or is thought of happening in a system [10]. The event signifies the activity.

In the context of BPM, events represent “state changes of objects within the context of a business process” [22]. Such objects comprise among other activities, actors, data elements, information systems, or entire processes operated in a BPM system. Another key idea of CEP is the aggregation of events. Through the definition of abstraction relationships it is possible to define complex events by aggregating a set of low-level events.

Here, CEP emerges to be a valuable instrument for a more timely analysis of process-related events on a business-level perspective.

As an independent system, CEP is “a parallel running platform that defines, analyses, processes events” [17].

Figure 3 introduces an architecture for event-driven business activity monitoring using Slipstream with CEP.

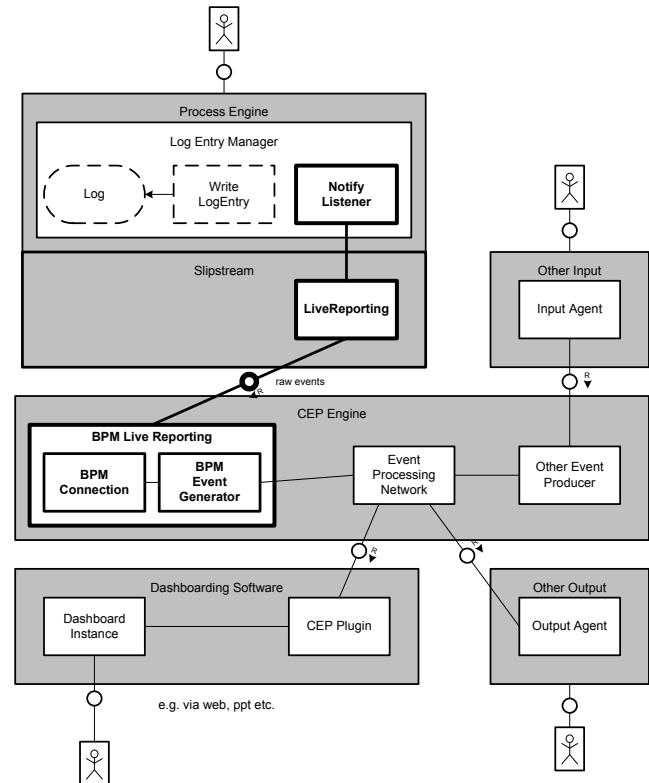


Figure 3: Implementation Option Slipstream CEP.

After an event has been detected and formatted, a corresponding message is sent to the CEP system. A typical CEP system comprises three logical components: an input adapter, an event processing network consisting of a set of connected event processing agents, and an output adapter.

The input adapter is responsible for transforming incoming events into the internal event format of the CEP system. Adapters are essential to configure a CEP system to run in a given IT landscape. Usually, there is an input adapter for each connected event producer and/ or each possible incoming event format. After adaptation, events are queued into an input event stream, ready to be processed by an event processing network.

Consequently, the *CEP Engine* will need to use an event producer plug-in to enable *BPM Live Reporting*. Communication can be done in a direct point-to-point fashion or via some kind of message-oriented middleware. Again, the use of a publish/ subscribe messaging mechanism is a promising way to realize a loosely coupled and scalable communication that allows for pushing events to processors.

While the systems are loosely coupled, the communication is not particular rigid or fragile as the model IDs of processes usually do not change when new versions are deployed and certain event types such as *process_ended* will always be picked up independent of a task that is inserted or deleted. Obviously, it is

desirable though that the BPM engine can generate and/ or update the CEP monitoring graph to increase the synchronicity of the two systems.

Event data sent in a specific format such as BPAF has to be decoded to be used in the CEP engine similar to the standalone approach. Again we need a component for the *BPM Connection*. In addition to that, the events received need to be translated into a format that is understood by the CEP engine's *Event Processing Network: A BPM Event Generator* to produce events.

With this kind of architecture, there are a number of other systems besides the BPM system which can act as *Input Agents* in the context of a business process. Sensors, for instance, can produce events that signify the actual execution of a business process activity or report on real-world events that are happening in the context of a business process. Also, transactional systems, such as ERP or CRM systems, can produce events that are related to a business process but lie outside of the control sphere of a BPM system (e.g. low stock alert, creation of a master data record). These raw events sent out by the various event producers are received and processed by a central event processor, i.e. the actual CEP system.

An event processing network (EPN) consists of a number of connected event processing agents (EPA). An EPA is an active component that monitors an event stream to detect and react on certain conditions. Its behavior is defined by a set of so-called event pattern rules. These reactive rules trigger on specific instances or patterns of input events. As a result of executing a rule, the agent creates output events and changes its local state. Three basic classes of EPAs can be distinguished [5]:

- A *filter EPA* performs filtering only. It passes on only those input events that are defined in its event pattern rules. A filter does not perform any transformation of events.
- A *transformation EPA* processes input events that are defined in its event pattern rules and creates output events that are functions of these input events. A transformation EPA can be stateless or stateful. A special case of transformation EPA is an aggregate EPA which takes as input a collection of events and creates a single derived event by applying a function over the input events. Other specializations of transformation EPAs are translate, enrich, project, split, and compose.
- A *pattern detect EPA* performs a pattern matching function on a stream of input events. It emits one or more derived events if it detects a combination of events in the input stream that satisfies a specified pattern.

Having defined a number EPAs, EPNs can be composed by linking EPAs so that the output of one agent is the input of another agent. The EPAs communicate by sending their output events to other agents. This allows for building complex event processing logic out of lightweight and reusable building blocks.

Also, EPNs can be composed into even larger networks of event processors for scalability or query distribution purposes. This way one EPN acts as the producer for another EPN which is the consumer.

After processing, events are queued into an output event stream, ready to be forwarded to possible event consumers. An event

output adapter is responsible for transforming the outgoing events into external formats that can be processed by the connected event consumers. After adaptation the outgoing data does not necessarily have to represent an event anymore; it can, for example, be in the form of metrics, messages or function.

An event consumer is an entity that receives and visualizes or acts upon events from a CEP system. It is the logical counterpart of the event producer. As depicted in Figure 3, *Dashboarding Software* is the main consumer in a business activity monitoring scenario. Dashboards receive the event data via a *CEP Plugin* which can communicate with the CEP engine and displays the data in a *Dashboard Instance*. Apart from dashboards, there are other options for *Output Agents* such as messaging via email, Twitter SMS or instant messenger.

Apart from being an event producer, BPM systems could also represent an event consumer. Based on the insights derived from processing events produced by a BPM system, an event processor might initiate actions that influence processes in that very system (or other BPM systems respectively). An event output adapter of a CEP system might, for instance, make an API call to start new processes or suspend, resume, or cancel running process instances. Likewise, gateways that control the flow of a process instance might be changed by manipulating or updating the conditions of a business rule.

4. CONCLUSIONS

Timely insight into a company's value chain is of great importance to be able to monitor and improve its operations. Business processes are the core building blocks of a company's value chain and provide a plethora of information that can be tapped into. However, traditional business process analytics is often only the second step in BPM. Consequently, real-time or at least near real-time insight into processes is far from being standardized. In the past, custom applications have been developed for this requirement. Recently, the topic of operational BI has gained attention through the emergence of more sophisticated CEP engines. However, the dedicated application to business process management is only in its infancy. Nevertheless, it promises cost saving due to the ability to react in a reasonable timeframe to errors, exceptions, or other indicators such as workload, waiting times, etc. Especially in a business network context it promises to facilitate a timely cross-boundary coordination of participants' activities.

We reviewed the current state of the art of BAM and CEP. Based on existing work, we developed an architecture for business activity monitoring to expose data from the wake of a process execution in real-time. We introduced an architecture that is capable of delivering real-time business activity monitoring but is extensible to produce complex events for various systems.

The architecture is software independent and can be instantiated with basically any combination BPM and CEP system. It also enables the monitoring and management of multiple BPM engines through its loose coupling and can also serve as the basis for future context-aware BPM systems.

We have implemented a proof of concept of this reference architecture using multiple SAP NetWeaver BPM servers as BPM engines, Corel8 as a CEP engine, and SAP BusinessObjects Xcelsius Enterprise as a visualization frontend. All of the above

can be downloaded for trial and evaluation and represent state-of-the-art BPM and CEP technology. Due to limitations in paper length we are not able to include more details on the successful validation of the reference architecture in this publication but will make it available during the presentation.

While implementation option 1 proved to be a simple and efficient option, it only allows for the definition of some standard KPI and changing them involves software development. It is also difficult to extend to a multi-BPM-engine scenario. Option 2 is very flexible and scalable. However, due to its flexibility, all functionality has to be built from scratch with CEP engines. Currently there is no *meta BAM engine* available which would facilitate using CEP for BAM.

In line with our CEP approach, current trends require BAM to extend processing functionality and call for a tighter alignment of BPM and CEP in business process analytics: There is (1) “the drive to provide richer types of observation which necessitates more event processing functions such as event pattern detection” and (2) “the need to provide more online observations when data comes from multiple event sources” [5]. However, even when this information is retrieved, “business processes remain static and cannot be changed dynamically to adapt to the actual scenario” [8] and therefore require guidance for translating insight into action.

So in addition to that, we plan to address several shortcomings in the area of business activity monitoring. Practically speaking, these include but are not limited to the lack of a common business event format, the lack of a standardized set of logging events, and the lack of standardized operations to trigger BPM engines for automated insight to action. On the conceptual side there is a clear need for an integrated or at least synchronized modeling paradigm for business processes and reporting on the one hand and business processes and their interaction with event processing networks on the other hand.

5. REFERENCES

- [1] Becker, J., Kugeler, M., and Rosemann, M. (eds.). 2003. *Process Management: A Guide for the Design of Business Processes*. Springer, Berlin.
- [2] Costello, C. and Molloy, O. 2008. Towards a Semantic Framework for Business Activity Monitoring and Management. In *Proceedings of the AAAI 2008 Spring Symposium* (Stanford, CA), K. Hinkelmann (ed.). 1-11.
- [3] DeFee, J. and Harmon, P. 2004. Business Activity Monitoring and Simulation. In *Workflow Handbook 2005*, L. Fischer (ed.). Future Strategies, Lighthouse Point, FL, 53-74.
- [4] Eckert, M. 2008. *Complex Event Processing with XChange EQ: Language Design, Formal Semantics, and Incremental Evaluation for Querying Events*. LMU München.
- [5] Etzion, O. and Niblett, P. 2010. *Event Processing in Action*. Manning Publications, Cincinnati, OH.
- [6] Few, S. 2004. Dashboard Confusion. Retrieved 2010-08-23 from <http://intelligent-enterprise.informationweek.com/showArticle.jhtml;jsessionid=BJXFCGOZCMUE3QE1GH PCKHWATMY32JVN?articleID=18300136>.
- [7] Hammer, M. and Champy, J. 1993. *Reengineering the Corporation: A Manifesto for Business Revolution*. HarperBusiness, New York, NY.
- [8] Hermosillo, G., Seinturier, L., and Duchien, L. 2010. Using Complex Event Processing for Dynamic Business Process Adaptation. In *Proceedings of the 7th IEEE International Conference on Service Computing (SCC)* (Miami, FL). 466-473.
- [9] Lauras, M., Marques, G., and Gourc, D. 2010. Towards a Multi-dimensional Project Performance Measurement System. *Decision Support Systems* 48, 2, 342-353.
- [10] Luckham, D. 2002. *The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems*. Addison-Wesley Professional, Boston, MA.
- [11] McCoy, D. W. 2002. Business Activity Monitoring: Calm Before the Storm. Gartner Research Note LE-15-9727.
- [12] Melchert, F., Winter, R., and Klesse, M. 2004. Aligning Process Automation and Business Intelligence to Support Corporate Performance Management. In *Proceedings of the 10th Americas Conference on Information Systems (AMCIS)* (New York, NY). 4053-4063.
- [13] Nesamoney, D. 2004. BAM: Event-driven Business Intelligence for the Real-Time Enterprise. *DM Review* 14, 3, 38-40.
- [14] Object Management Group Inc. 2006. Business Process Modeling Notation (BPMN) Specification 1.0.
- [15] Pedrinaci, C., Domingue, J., and de Medeiros, A. K. A. 2008. A Core Ontology for Business Process Analysis. In *Proceedings of the 5th European Semantic Web Conference (ESWC). Lecture Notes In Computer Science Vol. 5021* (Tenerife). 49-64.
- [16] van der Aalst, W. M. P., Reijers, H. A., and Weijters, A. 2007. Business Process Mining: An Industrial Application. *Information Systems* 32, 5, 713-732.
- [17] von Ammon, R., Ertlmaier, T., Etzion, O., Kofman, A., and Paulus, T. 2009. Integrating Complex Events for Collaborating and Dynamically Changing Business Processes. In *Proceedings of the 2nd Workshop on Monitoring, Adaptation and Beyond (MONA+)* (Stockholm). 1-16.
- [18] Wetzstein, B., Karastoyanova, D., Kopp, O., Leymann, F., and Zwink, D. 2010. Cross-Organizational Process Monitoring based on Service Choreographies. In *Proceedings of the ACM Symposium on Applied Computing (ACM SAC)* (Sierre). 2485-2490.
- [19] Workflow Management Coalition (WFMC). 1999. Workflow Management Coalition Terminology and Glossary. Retrieved 2010-08-23 from http://www.wfmc.org/standards/docs/TC-1011_term_glossary_v3.pdf.
- [20] Workflow Management Coalition (WFMC). 2008. Business Process Analytics Format. Draft Version 2.0. Retrieved 2010-08-23 from http://www.wfmc.org/index.php?option=com_docman&task=doc_download&gid=437&Itemid=72.
- [21] zur Muehlen, M. and Indulska, M. 2010. Modeling Languages for Business Processes and Business Rules: A Representational Analysis. *Information Systems* 35, 4, 379-390.
- [22] zur Muehlen, M. and Shapiro, R. 2010. Business Process Analytics. In *Handbook on Business Process Management: Strategic Alignment, Governance, People and Culture, Vol. 2*, J. vom Brocke and M. Rosemann (eds.). Springer, Berlin.